

W4111 – Introduction to Databases
Section 002, Fall 2025
Lecture 9: Module II
DBMS Architecture (3), NoSQL (3)



Introduction

Contents

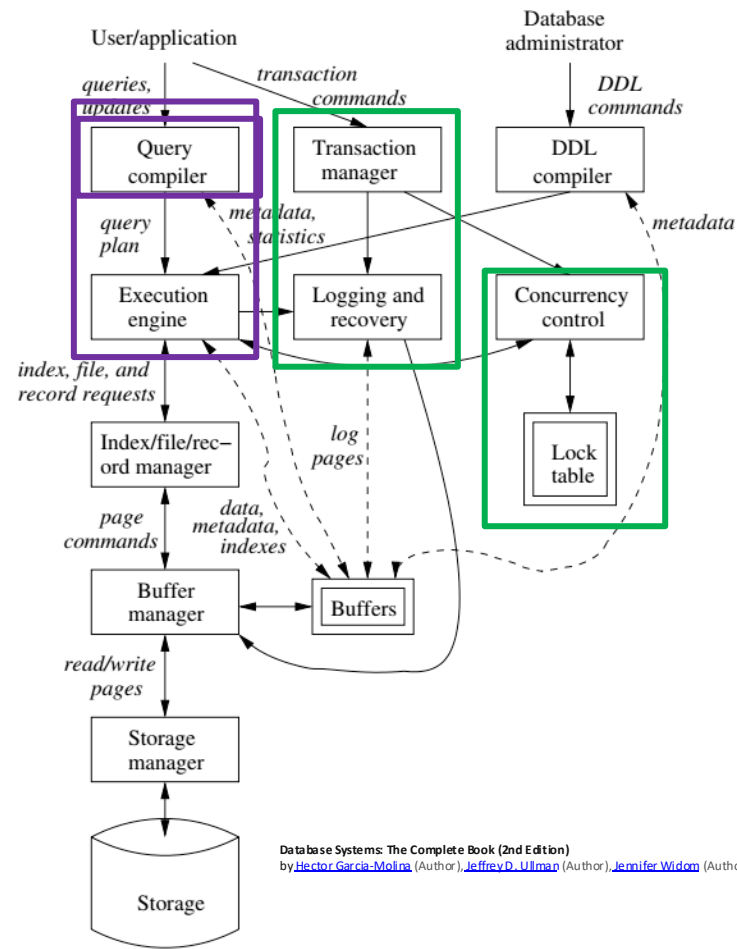
Content

Module II

Introduction and Reminder

Today

- Finishing
 - Query Optimization
- Starting
 - Transactions
 - Concurrency control
 - Logging and recovery

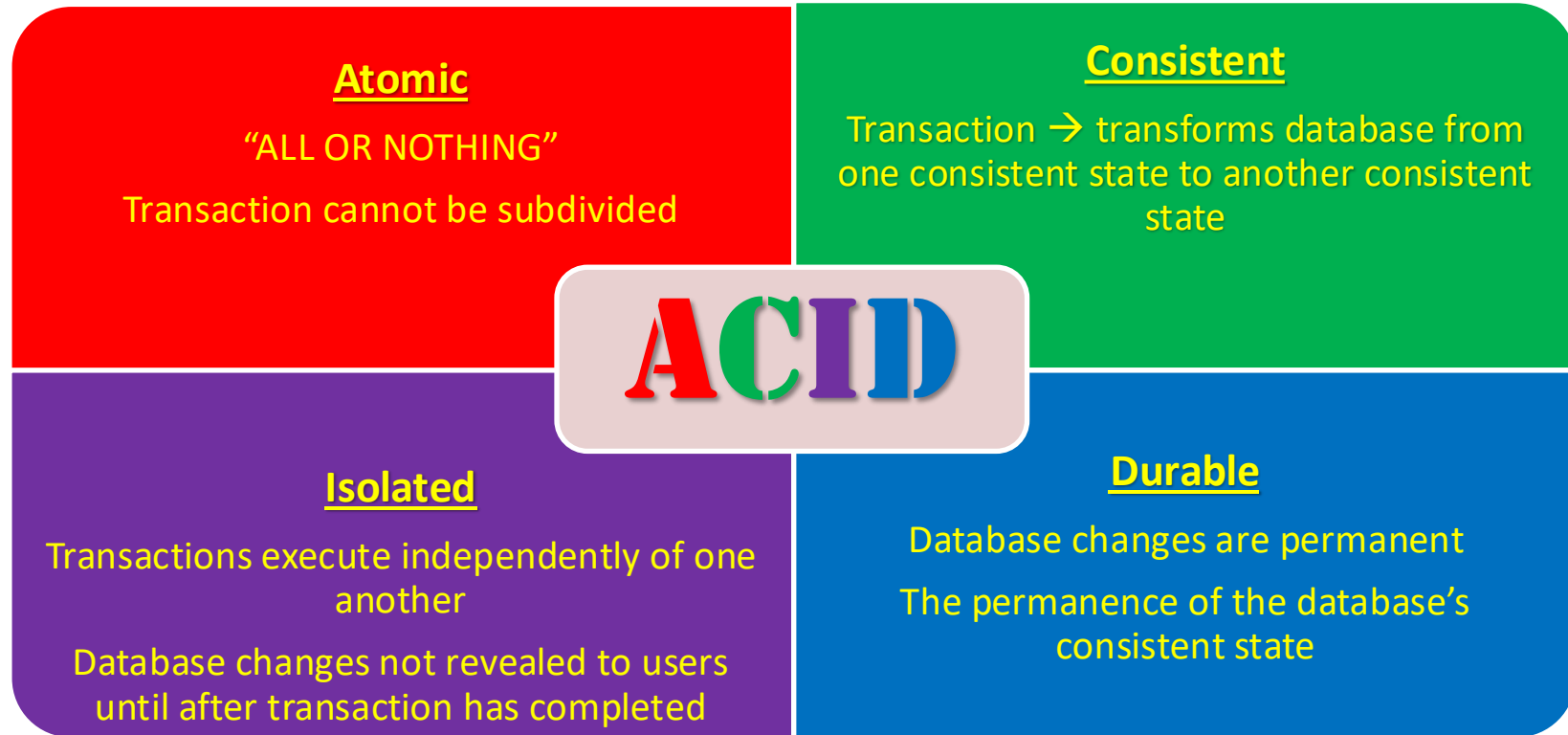


Transactions and Recovery

Reminder

Core Transaction Concept is ACID Properties

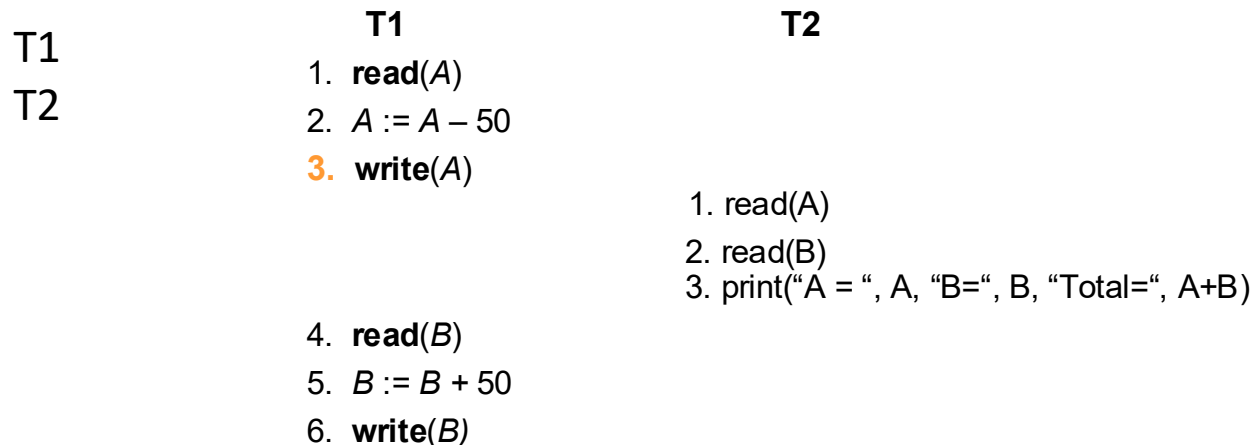
<http://slideplayer.com/slide/9307681>





Example of Fund Transfer (Cont.)

- T1, T2
- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

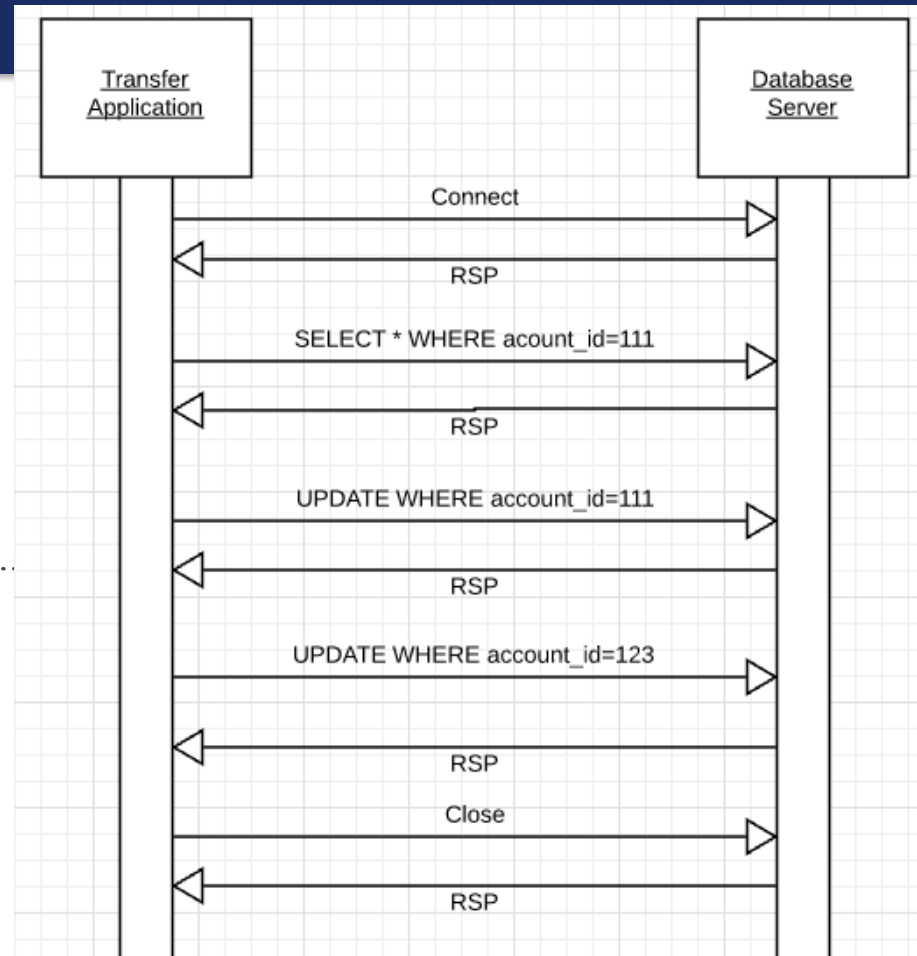


- Isolation can be ensured trivially by running transactions **serially**
 - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

Isolation

Isolation

- Transfer \$50 from
 - account_id=111 to
 - account_id=123
- Requires 3 SQL statements
 - SELECT from 111 to check balance $\geq$ \$50
 - UPDATE account_id=111
 - UPDATE account_id=123
- There are some interesting scenarios
 - Two different programs read the balance (\$51)
 - And decide removing \$50 is OK.
- DB constraints can prevent the conflict from happening, but ...
 - There are more complex scenarios that constraints do not prevent.
 - Not ALL databases support constraints.
 - The “correct” execution should be that
 - One transaction responds “insufficient funds”
 - Before attempting transfer instead of after attempting.



Isolation

- Consider two simultaneous transfer transactions T1 and T2.
- There are two equally **correct** executions
 1. T1 transfers, T2 responds “insufficient funds” and does not attempt transfer
 2. T2 transfers, T1 responds “insufficient funds” and does not attempt transfer
- Each correct simultaneous execution is equivalent to a serial (sequential) execution schedule
 - (1) Execute T1, Execute T2
 - (2) Execute T2, Execute T1
- NOTE:
 - We are focusing on correctness not
 - Fairness:
 - We do not care which transaction was actually submitted first.
 - And probably do not know due to networking, etc.

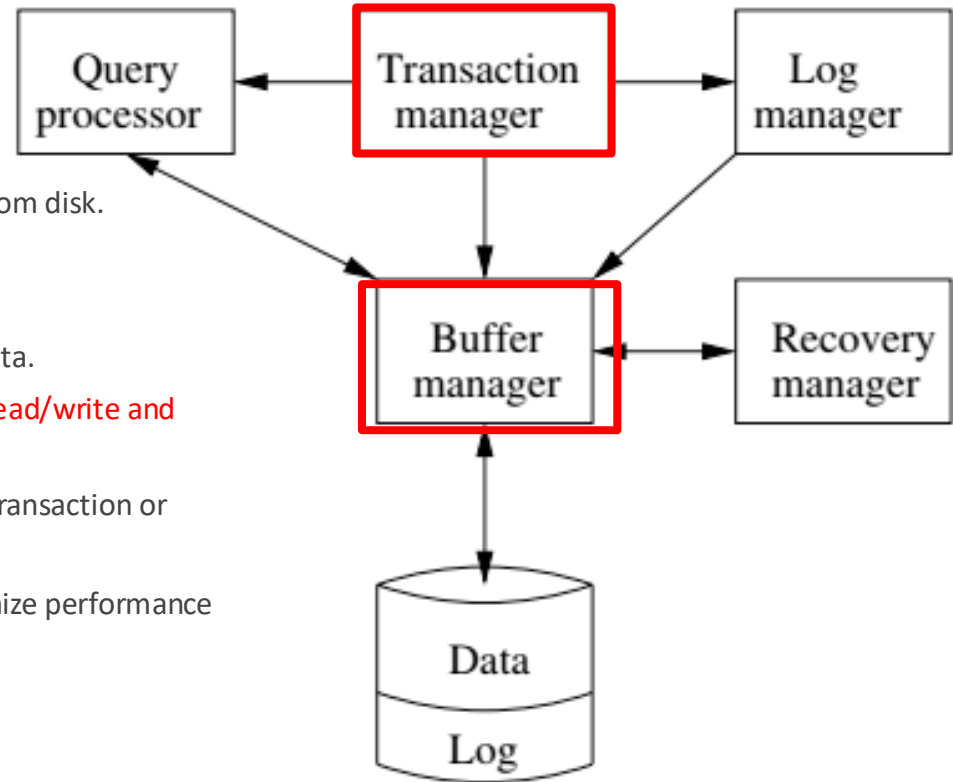
Serializability

“In [concurrency control](#) of [databases](#),^{[1][2]} [transaction processing](#) (transaction management), and various [transactional](#) applications (e.g., [transactional memory](#)^[3] and [software transactional memory](#)), both centralized and [distributed](#), a transaction [schedule](#) is **serializable** if its outcome (e.g., the resulting database state) is equal to the outcome of its transactions executed serially, i.e. without overlapping in time. Transactions are normally executed concurrently (they overlap), since this is the most efficient way. Serializability is the major correctness criterion for concurrent transactions' executions. It is considered the highest level of [isolation](#) between [transactions](#), and plays an essential role in [concurrency control](#). As such it is supported in all general purpose database systems.”
(<https://en.wikipedia.org/wiki/Serializability>)

DBMS ACID Implementation

Implementation Subsystems

- *Query processor* schedules and executes queries.
- *Buffer manager* controls reading/writing blocks/frames to/from disk.
- *Log manager* journals/records events to disk (log), e.g.
 - Transaction start/commit/abort
 - Transaction update of a block and previous value of data.
- *Transaction manager* coordinates query scheduling, buffer read/write and logging to ensure ACID.
- *Recovery manager* processes log to ensure ACID even after transaction or system failures.
- *Log* is a special type of block file used by the system to optimize performance and ensure ACID.



Garcia-Molina et al., p. 846



Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
 - **Increased processor and disk utilization**, leading to better transaction *throughput*
 - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
 - **Reduced average response time** for transactions: short transactions need not wait behind long ones.
- **Concurrency control schemes** – mechanisms to achieve isolation
 - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database
 - Will study in Chapter 15, after studying notion of correctness of concurrent executions.



Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
 - A schedule for a set of transactions must consist of all instructions of those transactions
 - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
 - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement



Schedule

18.1.1 Schedules

A *schedule* is a sequence of the important actions taken by one or more transactions. When studying concurrency control, the important read and write actions take place in the main-memory buffers, not the disk. That is, a database element A that is brought to a buffer by some transaction T may be read or written in that buffer not only by T but by other transactions that access A .

T_1	T_2
READ(A, t)	READ(A, s)
$t := t+100$	$s := s*2$
WRITE(A, t)	WRITE(A, s)
READ(B, t)	READ(B, s)
$t := t+100$	$s := s*2$
WRITE(B, t)	WRITE(B, s)

Figure 18.2: Two transactions

Garcia-Molina et al.



Schedule 1

- Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .
- A **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	



Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

- In Schedules 1, 2 and 3, the sum $A + B$ is preserved.



- | T_1 | T_2 |
|---|---|
| read (A)
$A := A - 50$ | read (A)
$temp := A * 0.1$
$A := A - temp$
write (A)
read (B) |
| write (A)
read (B)
$B := B + 50$
write (B)
commit | $B := B + temp$
write (B)
commit |



Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
 1. **Conflict serializability**
 2. **View serializability**

18.1.2 Serial Schedules

- Assume there are three
 - concurrently executing transactions
 - T1, T2 and T3
- The transaction manager
 - Enables concurrent execution
 - But schedules individual operations
 - To ensure that the final DB state
 - Is *equivalent* to one of the following schedules
 - T1, T2, T3
 - T1, T3, T2
 - T2, T1, T3
 - T2, T3, T1
 - T3, T1, T2
 - T3, T2, T1

Concurrent execution was *serializable*.

A schedule is *serial* if its actions consist of all the actions of one transaction, then all the actions of another transaction, and so on. No mixing of the actions is allowed.

T_1	T_2	A	B
		25	25
READ(A, t)			
t := t+100			
WRITE(A, t)		125	
READ(B, t)			
t := t+100			
WRITE(B, t)			125
	READ(A, s)		
	s := s*2		
	WRITE(A, s)	250	
	READ(B, s)		
	s := s*2		
	WRITE(B, s)		250

Figure 18.3: Serial schedule in which T_1 precedes T_2

Serializability (en.wikipedia.org/wiki/Serializability)

- **Serializability** is used to keep the data in the data item in a consistent state. Serializability is a property of a transaction [schedule](#) (history). It relates to the [isolation](#) property of a [database transaction](#).
- **Serializability** of a schedule means equivalence (in the outcome, the database state, data values) to a *serial schedule* (i.e., sequential with no transaction overlap in time) with the same transactions. It is the major criterion for the correctness of concurrent transactions' schedule, and thus supported in all general purpose database systems.
- **The rationale behind serializability** is the following:
 - If each transaction is correct by itself, i.e., meets certain integrity conditions,
 - then a schedule that comprises any *serial* execution of these transactions is correct (its transactions still meet their conditions):
 - "Serial" means that transactions do not overlap in time and cannot interfere with each other, i.e, complete *isolation* between each other exists.
 - Any order of the transactions is legitimate, (...)
 - As a result, a schedule that comprises any execution (not necessarily serial) that is equivalent (in its outcome) to any serial execution of these transactions, is correct.



Simplified view of transactions

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.



Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .
 1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 2. $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
 3. $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict
 4. $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.



Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule



Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6



Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

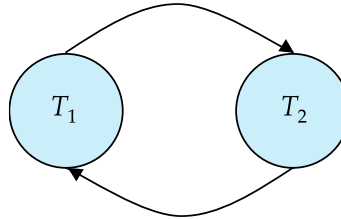
T_3	T_4
read (Q)	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.



Testing for Serializability

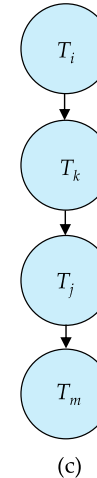
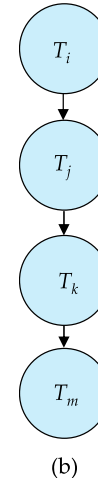
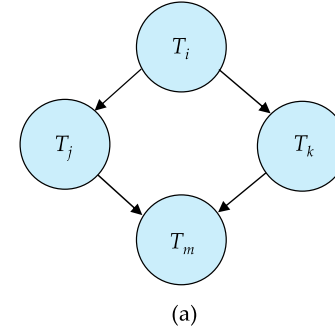
- Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$
- **Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph





Test for Conflict Serializability

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.
 - (Better algorithms take order $n + e$ where e is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
 - This is a linear order consistent with the partial order of the graph.
 - For example, a serializability order for Schedule A would be $T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$
 - Are there others?





Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.



Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	
abort		read (A)

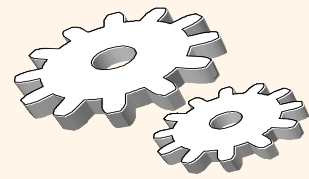
If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work



Cascadeless Schedules

- **Cascadeless schedules** — cascading rollbacks cannot occur;
 - For each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .
- Every Cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless



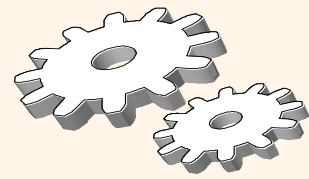
Lock-Based Concurrency Control

❖ Strict Two-phase Locking (Strict 2PL) Protocol:

- Each Xact must obtain a **S (shared) lock** on object before reading, and an **X (exclusive) lock** on object before writing.
- All locks held by a transaction are released when the transaction completes
 - **(Non-strict) 2PL Variant:** Release locks anytime, but cannot acquire locks after releasing any lock.
- If an Xact holds an X lock on an object, no other Xact can get a lock (S or X) on that object.

❖ Strict 2PL allows only serializable schedules.

- Additionally, it simplifies transaction aborts
- **(Non-strict) 2PL** also allows only serializable schedules, but involves more complex abort processing



Aborting a Transaction

- ❖ If a transaction T_i is aborted, all its actions have to be undone. Not only that, if T_j reads an object last written by T_i , T_j must be aborted as well!
- ❖ Most systems avoid such *cascading aborts* by releasing a transaction's locks only at commit time.
 - If T_i writes an object, T_j can read this only after T_i commits.
- ❖ In order to *undo* the actions of an aborted transaction, the DBMS maintains a *log* in which every write is recorded. This mechanism is also used to recover from system crashes: all active Xacts at the time of the crash are aborted when the system comes back up.

MySQL (Locking) Isolation

13.3.6 SET TRANSACTION Syntax

```
1  SET [GLOBAL | SESSION] TRANSACTION
2      transaction_characteristic [, transaction_characteristic] ...
3
4  transaction_characteristic:
5      ISOLATION LEVEL level
6      | READ WRITE
7      | READ ONLY
8
9  level:
10     REPEATABLE READ
11     | READ COMMITTED
12     | READ UNCOMMITTED
13     | SERIALIZABLE
```

Scope of Transaction Characteristics

You can set transaction characteristics globally, for the current session, or for the next transaction:

- With the `GLOBAL` keyword, the statement applies globally for all subsequent sessions. Existing sessions are unaffected.
- With the `SESSION` keyword, the statement applies to all subsequent transactions performed within the current session.
- Without any `SESSION` or `GLOBAL` keyword, the statement applies to the next (not started) transaction performed within the current session. Subsequent transactions revert to using the `SESSION` isolation level.

Isolation Levels

([https://en.wikipedia.org/wiki/Isolation_\(database_systems\)](https://en.wikipedia.org/wiki/Isolation_(database_systems)))

Not all transaction use cases require 2PL and serializable execution. Databases support a set of levels.

- **Serializable**

- With a lock-based [concurrency control](#) DBMS implementation, [serializability](#) requires read and write locks (acquired on selected data) to be released at the end of the transaction. Also *range-locks* must be acquired when a [SELECT](#) query uses a ranged *WHERE* clause, especially to avoid the [phantom reads](#) phenomenon.
- *The execution of concurrent SQL-transactions at isolation level SERIALIZABLE is guaranteed to be serializable. A serializable execution is defined to be an execution of the operations of concurrently executing SQL-transactions that produces the same effect as some serial execution of those same SQL-transactions. A serial execution is one in which each SQL-transaction executes to completion before the next SQL-transaction begins.*

- **Repeatable reads**

- In this isolation level, a lock-based [concurrency control](#) DBMS implementation keeps read and write locks (acquired on selected data) until the end of the transaction. However, *range-locks* are not managed, so [phantom reads](#) can occur.
- Write skew is possible at this isolation level, a phenomenon where two writes are allowed to the same column(s) in a table by two different writers (who have previously read the columns they are updating), resulting in the column having data that is a mix of the two transactions.^{[3][4]}

- **Read committed**

- In this isolation level, a lock-based [concurrency control](#) DBMS implementation keeps write locks (acquired on selected data) until the end of the transaction, but read locks are released as soon as the [SELECT](#) operation is performed (so the [non-repeatable reads phenomenon](#) can occur in this isolation level). As in the previous level, *range-locks* are not managed.
- Putting it in simpler words, read committed is an isolation level that guarantees that any data read is committed at the moment it is read. It simply restricts the reader from seeing any intermediate, 'dirty' read. It makes no promise whatsoever that if the transaction re-issues the read, it will find the same data; data is free to change after it is read.

- **Read uncommitted**

- This is the *lowest* isolation level. In this level, [dirty reads](#) are allowed, so one transaction may see *not-yet-committed* changes made by other transactions



Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
 - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
 - E.g., database statistics computed for query optimization can be approximate (why?)
 - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance



Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read.
 - Repeated reads of same record must return same value.
 - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read.
 - Successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.



Levels of Consistency

- Lower degrees of consistency useful for gathering approximate information about the database
- Warning: some database systems do not ensure serializable schedules by default
- E.g., Oracle (and PostgreSQL prior to version 9) by default support a level of consistency called snapshot isolation (not part of the SQL standard)

In Databases, Cursors Define *Isolation*

- We have talked about ACID transactions

Isolation level	Dirty reads	Non-repeatable reads	Phantoms
Read Uncommitted	may occur	may occur	may occur
Read Committed	-	may occur	may occur
Repeatable Read	-	-	may occur
Serializable	-	-	-

```
InitialContext ctx = new InitialContext();
DataSource ds = (DataSource)
ctx.lookup("jdbc/MyBase");
Connection con = ds.getConnection();
DatabaseMetaData dbmd = con.getMetaData();
if (dbmd.supportsTransactionIsolationLevel(TRANSACTION_SERIALIZABLE)
{ Connection.setTransactionIsolation(TRANSACTION_SERIALIZABLE); }
```

- Isolation
 - Determines what happens when two or more threads are manipulating the data at the same time.
 - And is defined relative to where cursors are and what they have touched.
 - Because the cursor movement determines *what you are reading or have read*.
- *But, ... Cursors are client conversation state and cannot be used in REST.*

ACID, BASE CAP Theorem

Eventual Consistency

- “Eventual consistency is a consistency model used in distributed computing to achieve high availability. Put simply: if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value.[1] Eventual consistency, also called optimistic replication, is widely deployed in distributed systems and has origins in early mobile computing projects. A system that has achieved eventual consistency is often said to have converged, or achieved replica convergence. Eventual consistency is a weak guarantee – most stronger models, like linearizability, are trivially eventually consistent.
- Concepts and motivation:
 - Availability (and scalability) require multiple DBMS.
 - Scalability requires multiple copies of each data item.
 - Strict consistency requires complex, distributed locking and transaction protocols.
 - Failure precludes the ability to lock.
 - *Most* of the time, conflicts do not occur or are not crucial.
 - When conflicts occur, much of the time, it is possible to reconcile conflicts instead of preventing them.

BASE

Eventually-consistent services are often classified as providing BASE (Basically Available, Soft state, Eventual consistency) semantics, in contrast to traditional ACID (Atomicity, Consistency, Isolation, Durability) guarantees. Rough definitions of each term in BASE:

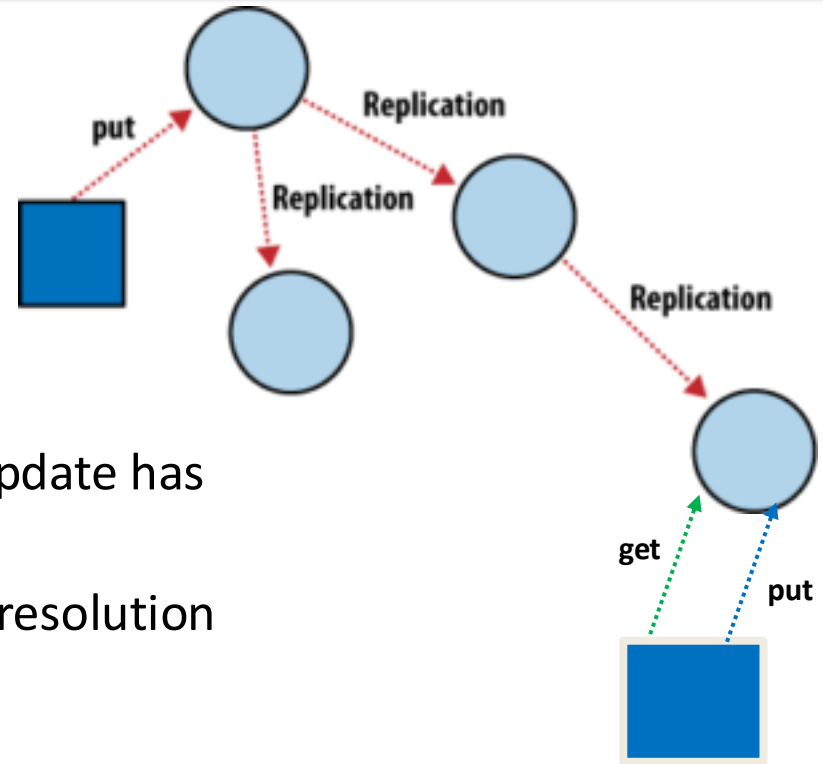
- **Basically Available:** basic reading and writing operations are available as much as possible (using all nodes of a database cluster), but without any kind of consistency guarantees (the write may not persist after conflicts are reconciled, the read may not get the latest write)
- **Soft state:** without consistency guarantees, after some amount of time, we only have some probability of knowing the state, since it may not yet have converged
- **Eventually consistent:** If the system is functioning and we wait long enough after any given set of inputs, we will eventually be able to know what the state of the database is, and so any further reads will be consistent with our expectations

ACID – BASE (Simplistic Comparison)

ACID (relational)	BASE (NoSQL)
Strong consistency	Weak consistency
Isolation	Last write wins (Or other strategy)
Transaction	Program managed
Robust database	Simple database
Simple code (SQL)	Complex code
Available and consistent	Available and partition-tolerant
Scale-up (limited)	Scale-out (unlimited)
Shared (disk, mem, proc etc.)	Nothing shared (parallellizable)

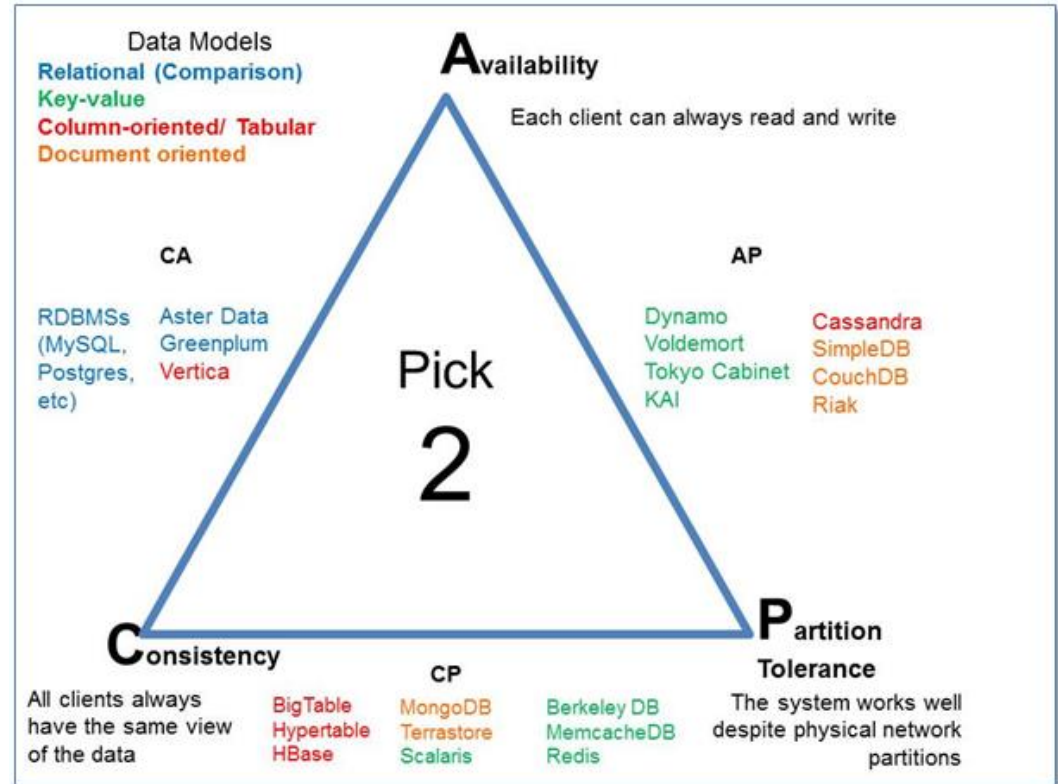
Eventual Consistency

- Availability and scalability via
 - Multiple, replicated data stores.
 - Read goes to “any” replica.
 - PUT/POST/DELETE
 - Goes to any replica
 - Change propagate asynchronously
- GET may not see the latest value if the update has not propagated to the replica.
- There are several algorithms for conflict resolution
 - Detect and handle in application.
 - Clock/change vectors/version numbers
 -



CAP Theorem

- **Consistency**
Every read receives the most recent write or an error.
- **Availability**
Every request receives a (non-error) response – without guarantee that it contains the most recent write.
- **Partition Tolerance**
The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes



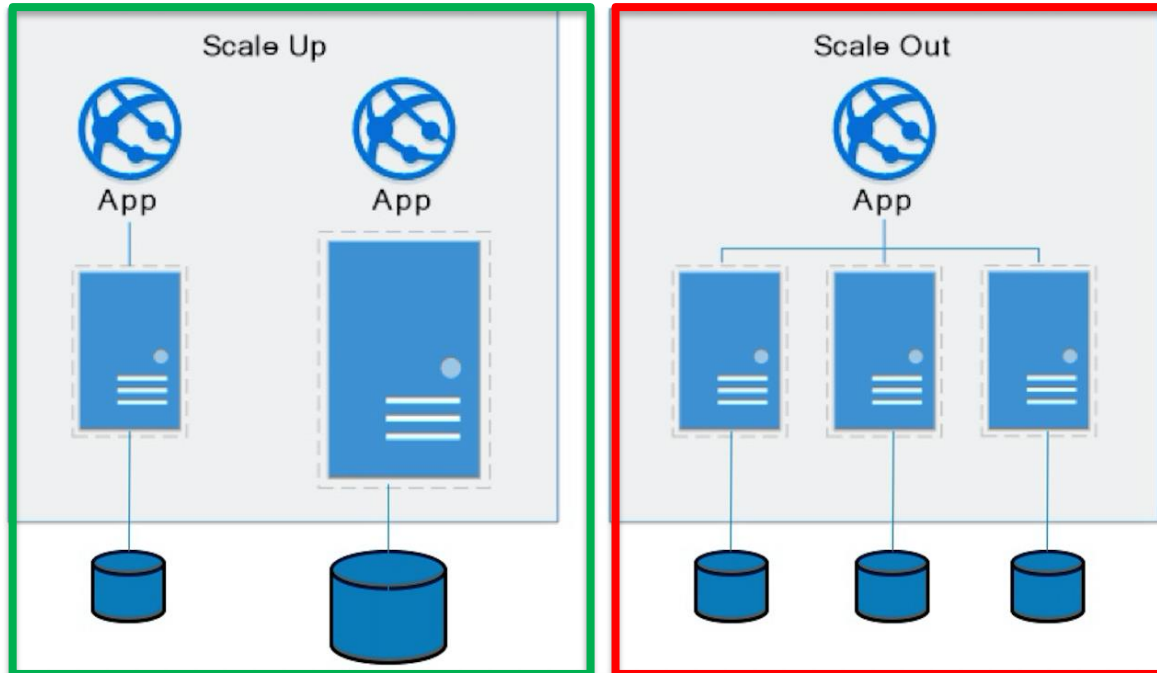
Scalability Availability

Scalability

Scalability is the property of a system to handle a growing amount of work by adding resources to the system.

Replace system with a bigger machine,
e.g. more memory, CPU,

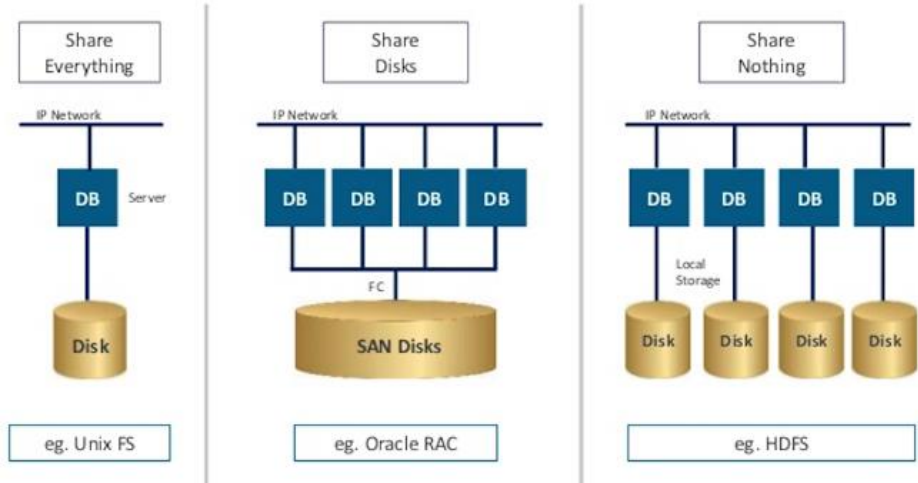
Add another system.



- **Scale-up:**
 - Less incremental.
 - More disruptive.
 - More expensive for extremely large systems.
 - Does not improve availability
- **Scale-out:**
 - Incremental cost.
 - Data replication enables availability.
 - Does not work well for functions like JOIN, referential integrity,

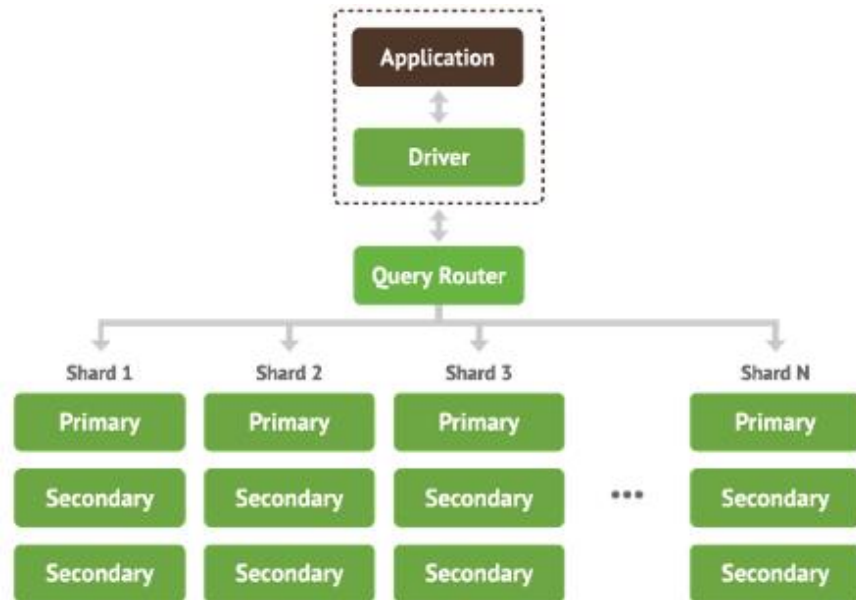
Disk Architecture for Scale-Out

- Share disks:
 - Is basically scale-up for data/disks. You can use NAS, SAN and RAID.
 - Isolation/Integrity requires distributed locking to control access from multiple database servers.
- Share nothing:
 - Is basically scale-out for disks.
 - Data is partitioned into *shards* based on a function $f()$ applied to a key.
 - Can improve availability, at the code consistency, with data replication.
 - There is a router that sends requests to the proper shard based on the function.

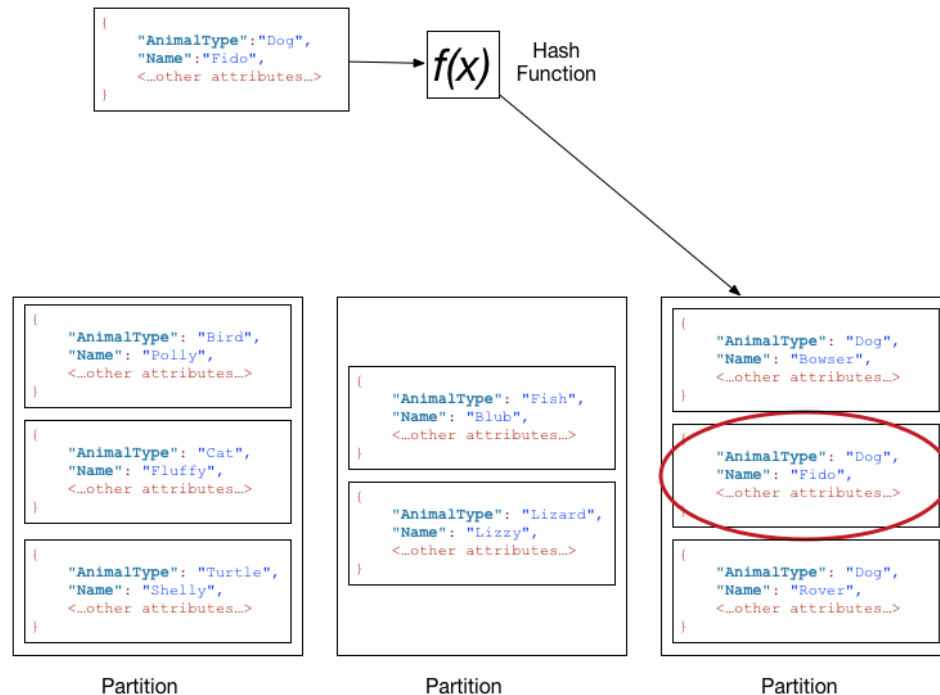


Shared Nothing, Scale-Out

MongoDB Sharding



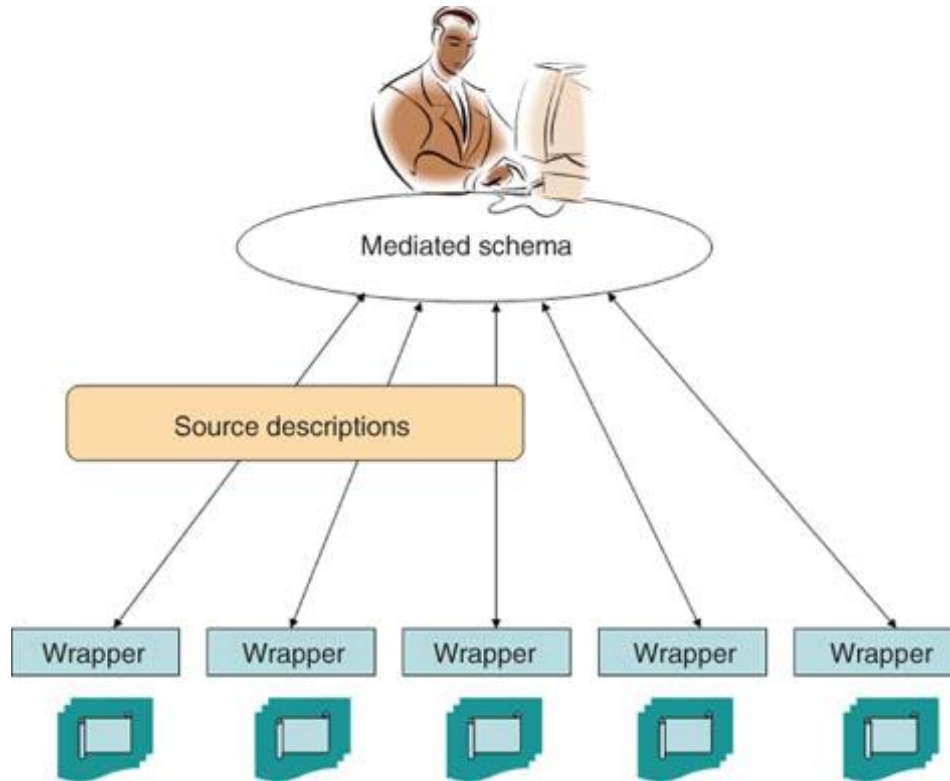
DynamoDB Partitioning



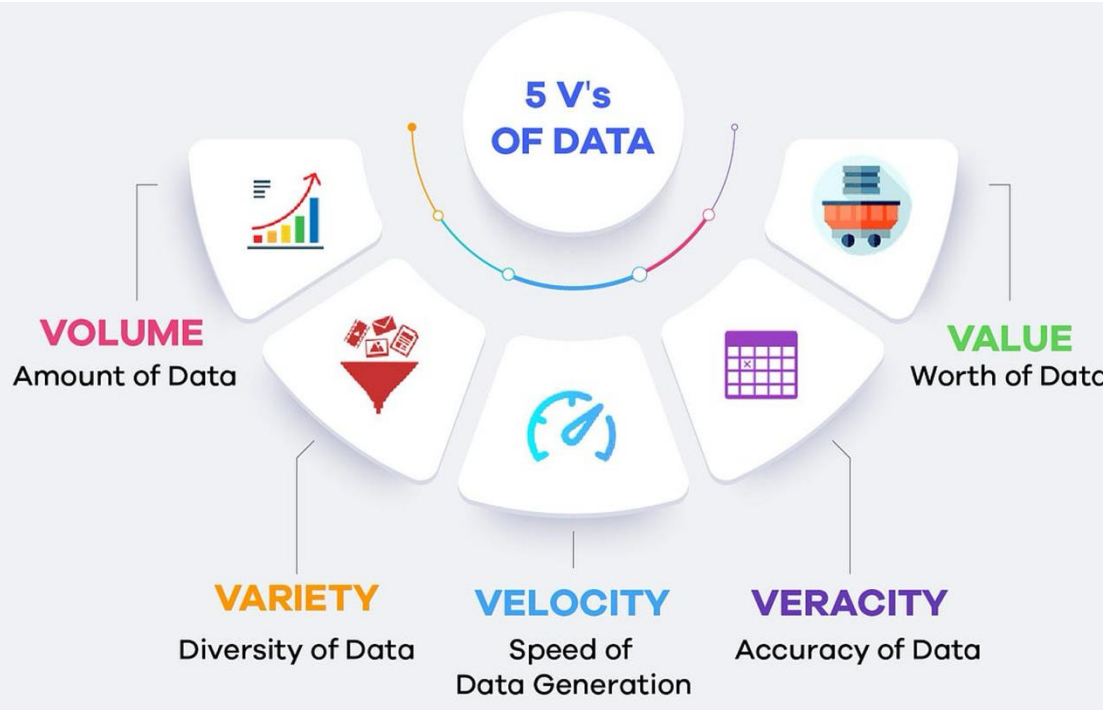
Big Data

Some Core Concepts

Enterprise Information Integration



5 V's of Big Data



https://medium.com/@get_excelsior/big-data-explained-the-5v-s-of-data-ae80cbe8ded1

Types Of Big Data

Following are the types of Big Data:

1. **Structured**
2. **Unstructured**
3. **Semi-structured**

Structured

Any data that can be stored, accessed and processed in the form of fixed format is termed as a 'structured' data. Over the period of time, talent in computer science has achieved greater success in developing techniques for working with such kind of data (where the format is well known in advance) and also deriving value out of it. However, nowadays, we are foreseeing issues when a size of such data grows to a huge extent, typical sizes are being in the range of multiple zettabytes.

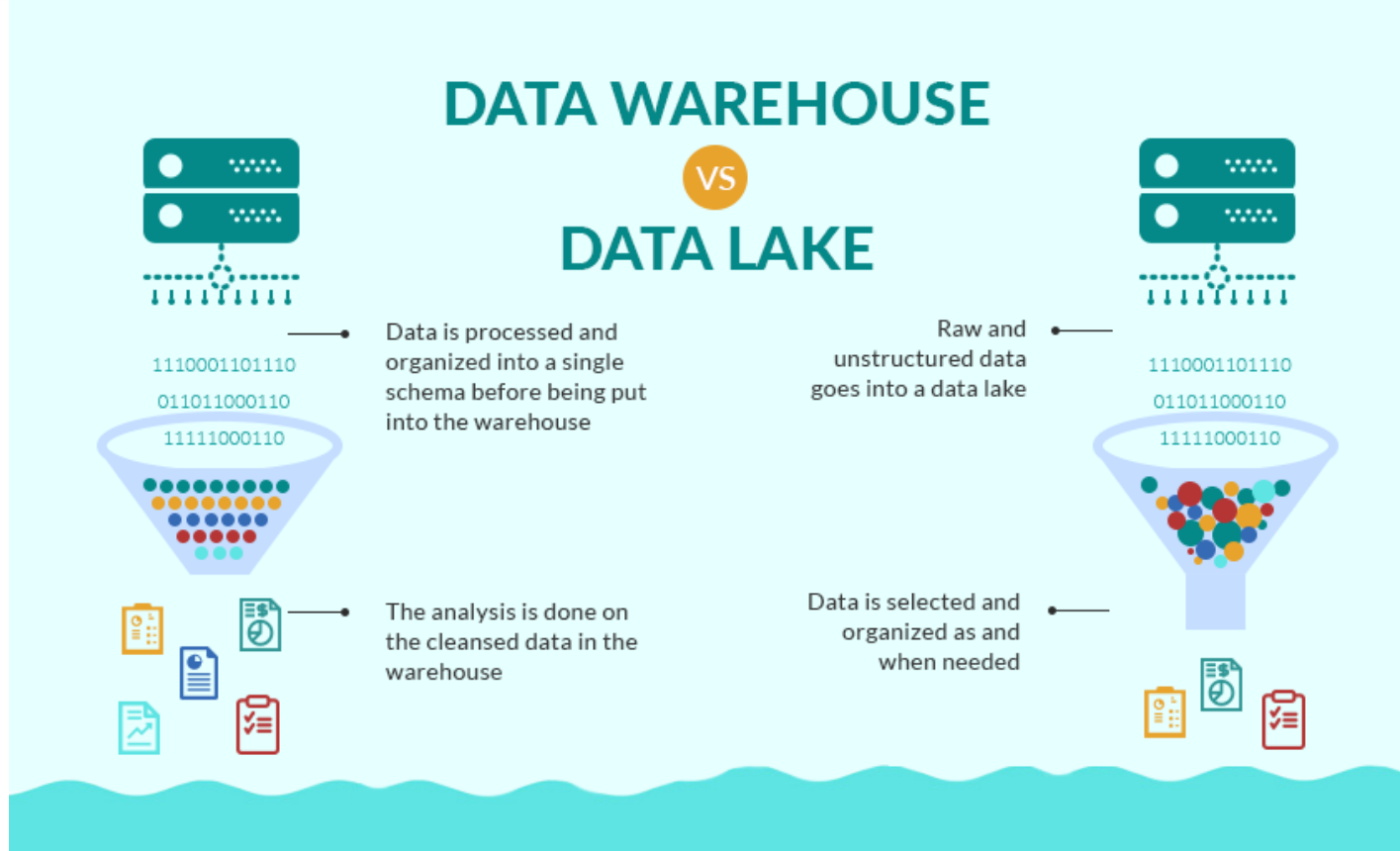
Unstructured

Any data with unknown form or the structure is classified as unstructured data. In addition to the size being huge, un-structured data poses multiple challenges in terms of its processing for deriving value out of it. A typical example of unstructured data is a heterogeneous data source containing a combination of simple text files, images, videos etc. Now day organizations have wealth of data available with them but unfortunately, they don't know how to derive value out of it since this data is in its raw form or unstructured format.

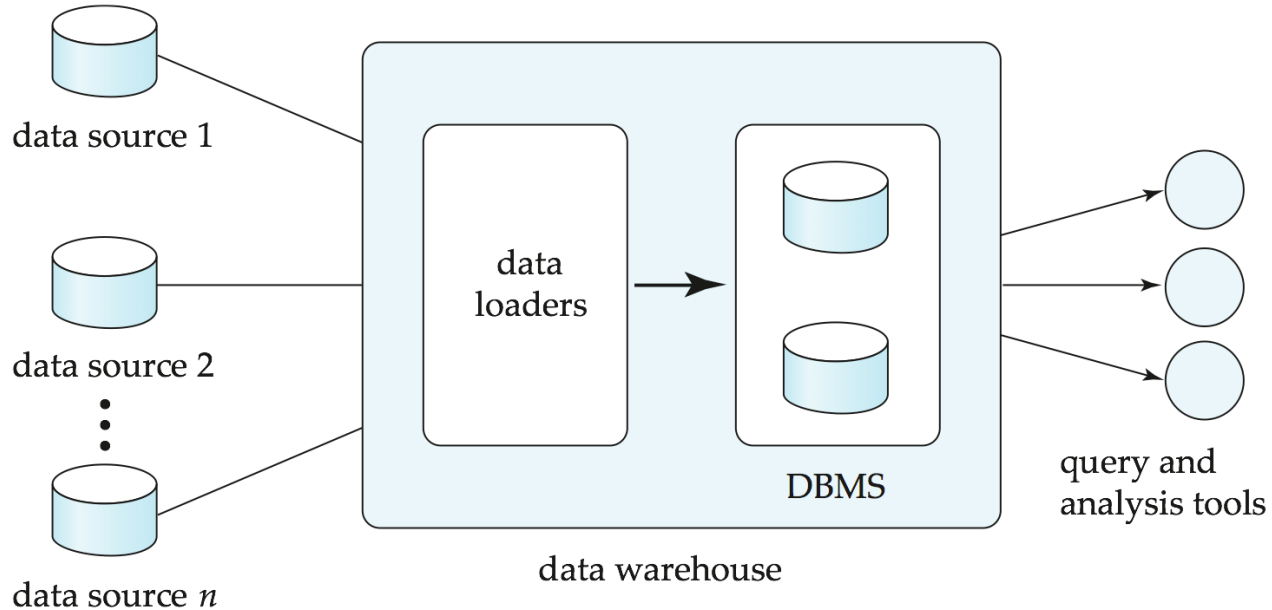
Semi-structured

Semi-structured data can contain both the forms of data. We can see semi-structured data as a structured in form but it is actually not defined with e.g. a table definition in relational **DBMS**. Example of semi-structured data is a data represented in an XML file.

Data Warehouse and Data Lake



Data Warehousing

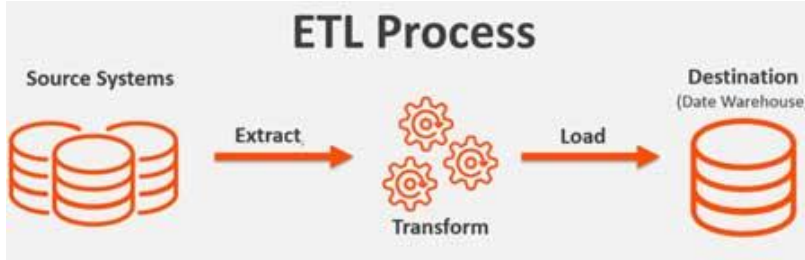


Overview (Cont.)

- Common steps in data analytics
 - Gather data from multiple sources into one location
 - Data warehouses also integrated data into common schema
 - Data often needs to be **extracted** from source formats, **transformed** to common schema, and **loaded** into the data warehouse
 - Can be done as **ETL (extract-transform-load)**, or **ELT (extract-load-transform)**
 - Generate aggregates and reports summarizing data
 - Dashboards showing graphical charts/reports
 - **Online analytical processing (OLAP) systems** allow interactive querying
 - Statistical analysis using tools such as R/SAS/SPSS
 - Including extensions for parallel processing of big data
 - Build **predictive models** and use the models for decision making

ETL Concepts

<https://databricks.com/glossary/extract-transform-load>



Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three**

Data Extraction methods:

1. Partial Extraction – The easiest way to obtain the data is if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification – Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

Transform

Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

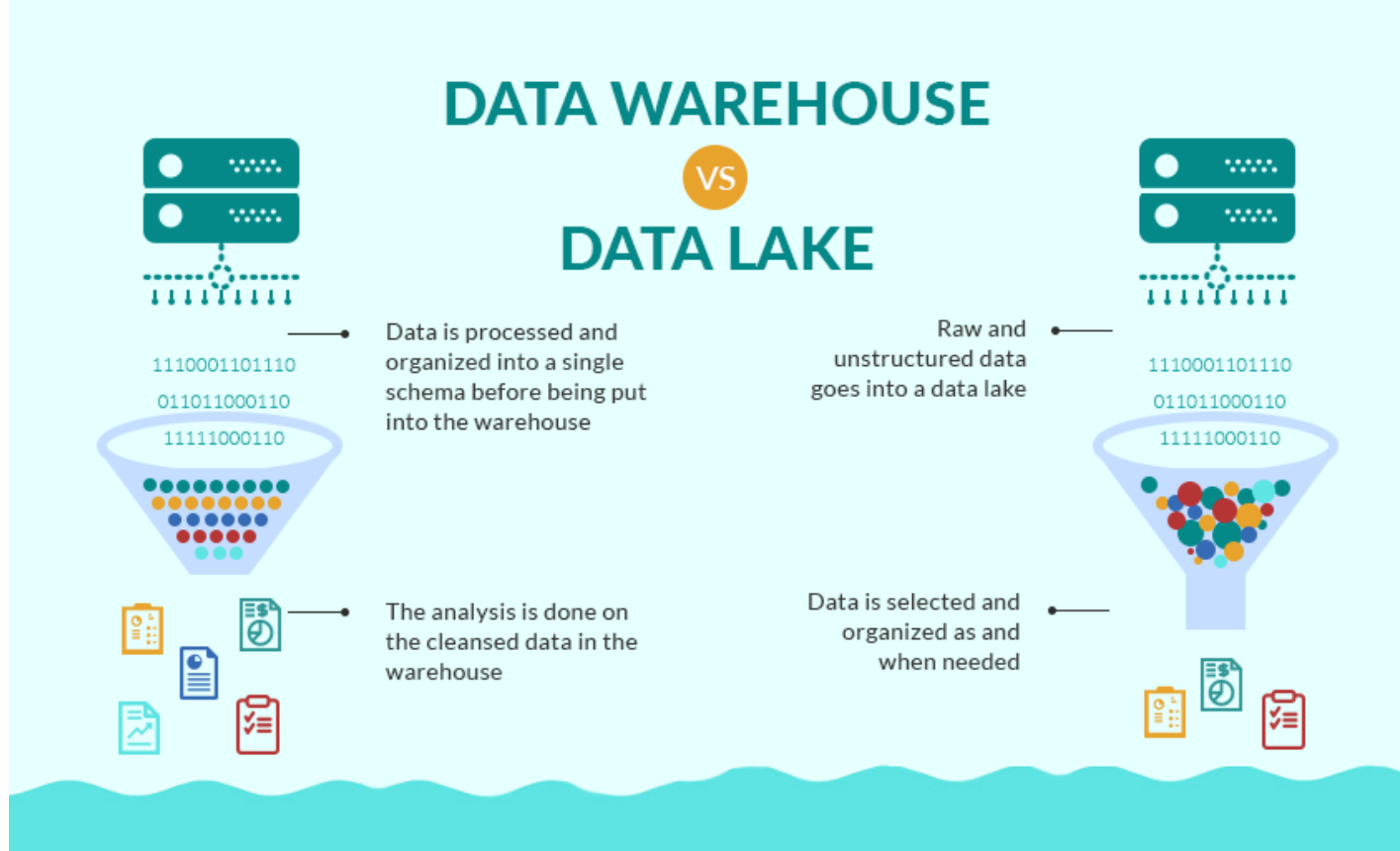
- Basic transformations:
 - Cleaning
 - Format revision
 - Data threshold validation checks
 - Restructuring
 - Deduplication
- Advanced transformations:
 - Filtering
 - Merging
 - Splitting
 - Derivation
 - Summarization
 - Integration
 - Aggregation
 - Complex data validation

Load

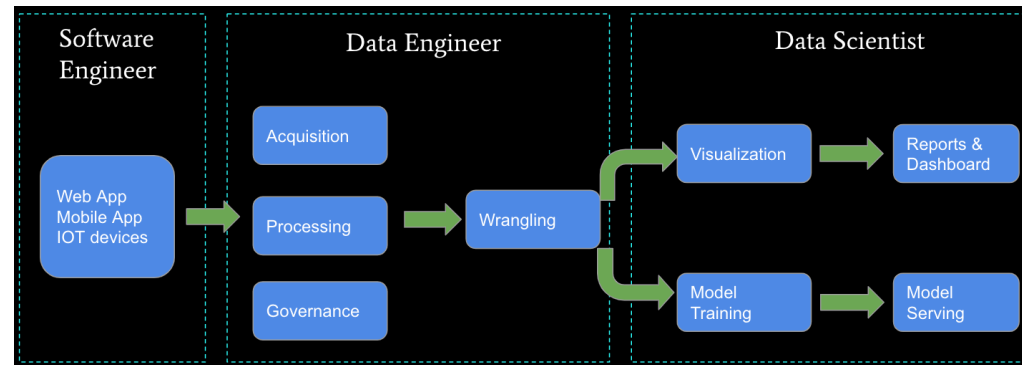
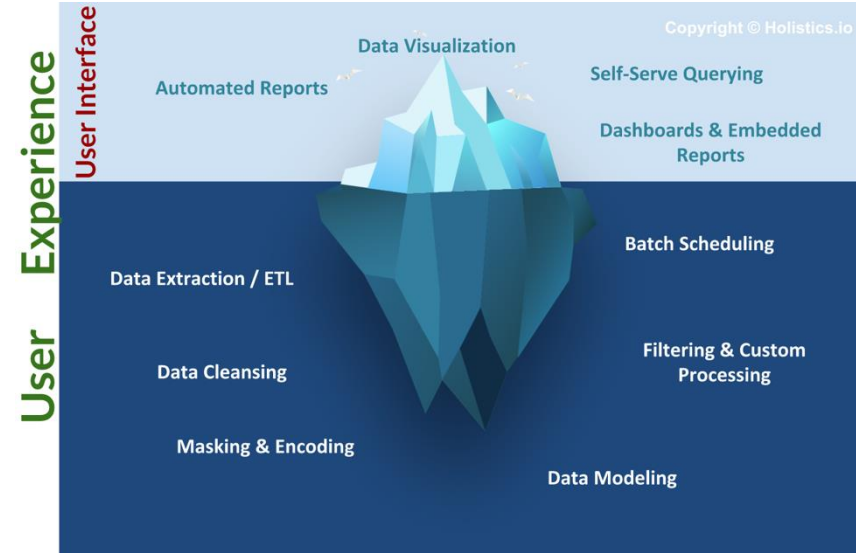
Finally, the load function is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

Data Engineering

Data Warehouse and Data Lake



Business Intelligence, Insight, Analysis,

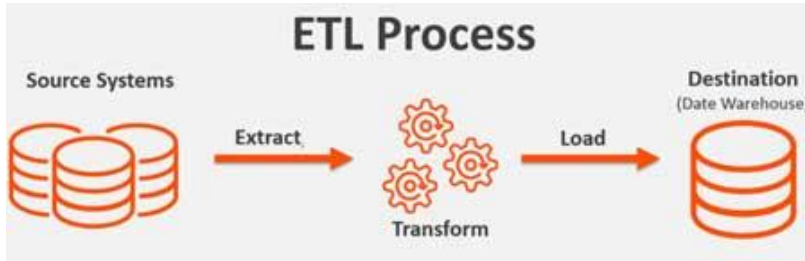


- The “fun” stuff in data science and AI/ML is the “tip of the iceberg.”
- Data engineering is a necessary condition for producing analyzable data. This is often more than 80% of the hard work.
- We will do some small data engineering projects in this course.



ETL Concepts

<https://databricks.com/glossary/extract-transform-load>



Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three**

Data Extraction methods:

1. Partial Extraction – The easiest way to obtain the data is if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification – Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

Transform

Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

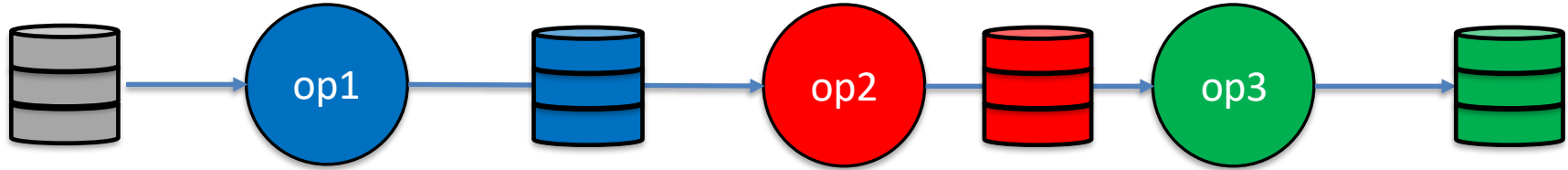
- Basic transformations:
 - Cleaning
 - Format revision
 - Data threshold validation checks
 - Restructuring
 - Deduplication
- Advanced transformations:
 - Filtering
 - Merging
 - Splitting
 - Derivation
 - Summarization
 - Integration
 - Aggregation
 - Complex data validation

Load

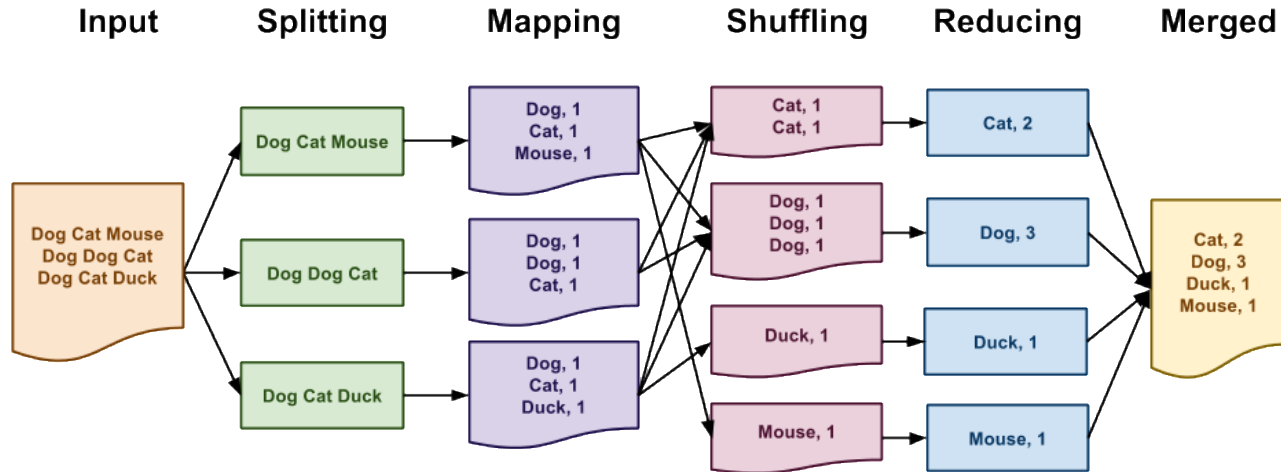
Finally, the load function is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

MapReduce

MapReduce is a data flow program with relatively simple operators on the data set.



With each operator implemented in parallel on multiple nodes for performance.



What we want more complex “operators?”

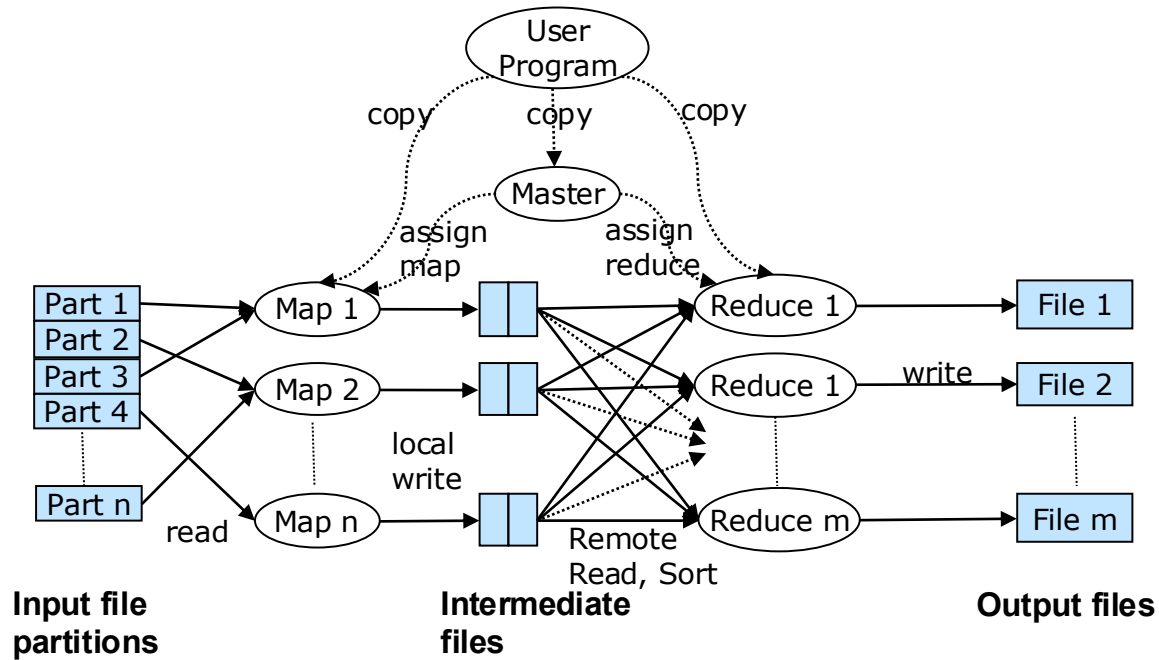


Map Reduce vs. Databases

- Map Reduce widely used for parallel processing
 - Google, Yahoo, and 100' s of other companies
 - Example uses: compute PageRank, build keyword indices, do data analysis of web click logs,
 - Allows procedural code in map and reduce functions
 - Allows data of any type
- Many real-world uses of MapReduce cannot be expressed in SQL
- But many computations are much easier to express in SQL
 - Map Reduce is cumbersome for writing simple queries



Parallel Processing of MapReduce Job





Map Reduce vs. Databases (Cont.)

- Relational operations (select, project, join, aggregation, etc.) can be expressed using Map Reduce
- SQL queries can be translated into Map Reduce infrastructure for execution
 - Apache Hive SQL, Apache Pig Latin, Microsoft SCOPE
- Current generation execution engines support not only Map Reduce, but also other algebraic operations such as joins, aggregation, etc. natively.

The Value of Blocks

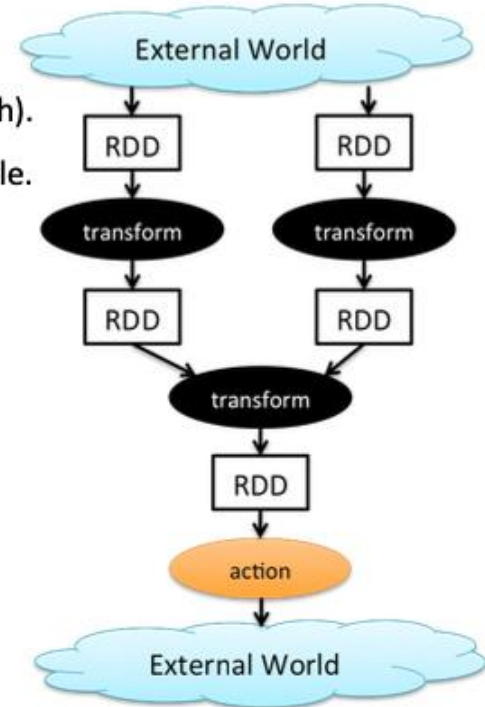
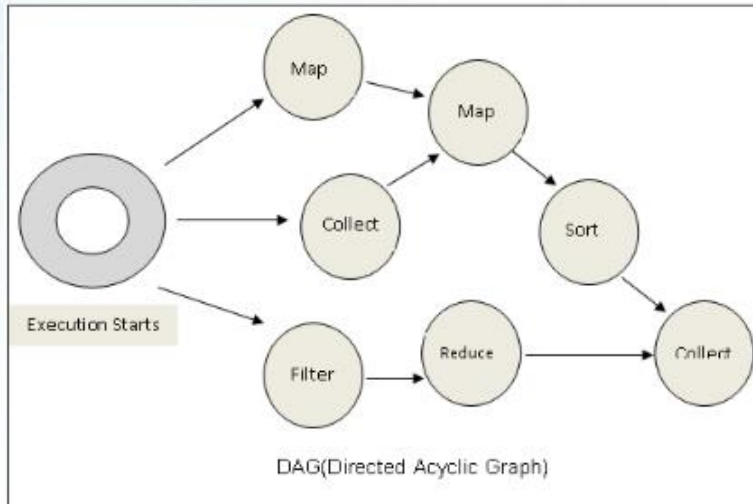
- The “normal” file system abstraction is a stream.
- Consider a very large CSV file:
 - We can read using the normal file system functions and APIs,
 - But we have no idea where records start/end, and
 - Which records on on which blocks.
 - ➔
 - Parallel processing of data in a file is very hard using the stream model.
- Databases, big data tools, etc. surface concepts of record size, block I/Os, mapping of and organization of records to blocks, block location, ➔
 - Can exploit multiple processors to parallel process large datasets.
 - Organize the data to optimize processing, file I/Os, etc.

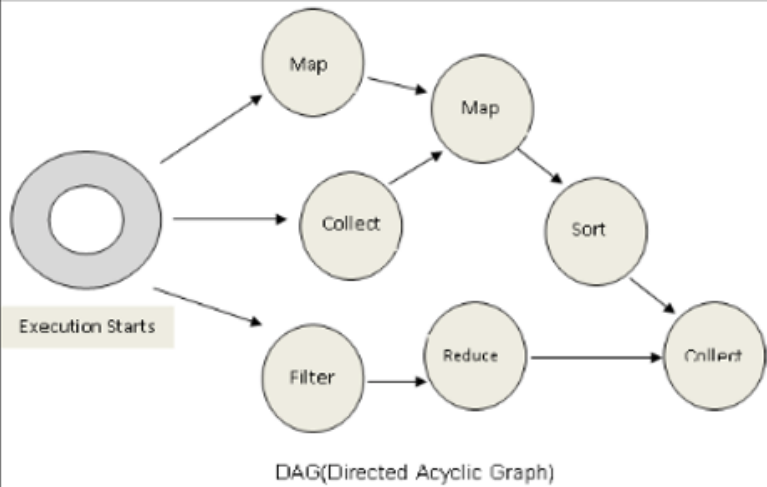
Algebraic Operations

Algebraic Operations

- Current generation execution engines
 - Natively support algebraic operations such as joins, aggregation, etc. natively.
 - Allow users to create their **own algebraic operators**
 - Support trees of algebraic operators that can be executed on **multiple nodes in parallel**
- E.g. Apache Tez, Spark
 - Tez provides low level API; Hive on Tez compiles SQL to Tez
 - Spark provides more user-friendly API
- In the relational model,
 - They are relations.
 - A fixed set of operators that produce relations from relations (closed).
 - Are declarative languages and compute the execution graph/plan.
- The Spark/... engines:
 - RDDs, Data Frames
 - Enable programmers to develop and add new operators.
 - Operators convert reliable distributed datasets/data frames to new RDDs/data frames.
 - Data engineer explicitly defines the execution graph (data flow).

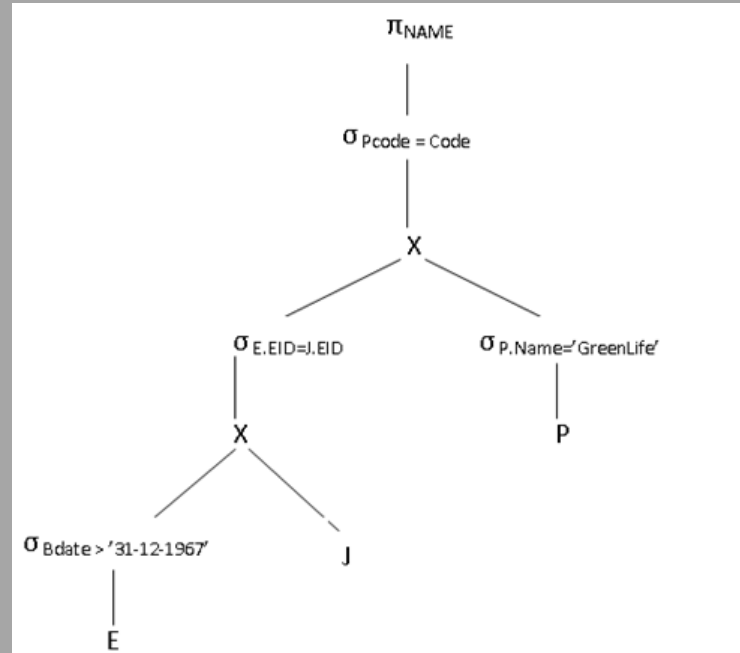
- All jobs in spark comprise a series of operators and run on a set of data.
- All the operators in a job are used to construct a DAG (Directed Acyclic Graph).
- The DAG is optimized by rearranging and combining operators where possible.





Conceptually similar to query evaluation graph, but ...

- You explicitly define the graph.
- You can develop your own operators.



Algebraic Operations in Spark

- **Resilient Distributed Dataset (RDD)** abstraction
 - Collection of records that can be stored across multiple machines
- RDDs can be created by applying algebraic operations on other RDDs
- RDDs can be lazily computed when needed
- Spark programs can be written in Java/Scala/R
 - Our examples are in Java
- Spark makes use of Java 8 Lambda expressions; the code
`s -> Arrays.asList(s.split(" ")).iterator()`
defines unnamed function that takes argument `s` and executes the expression `Arrays.asList(s.split(" ")).iterator()` on the argument
- Lambda functions are particularly convenient as arguments to map, reduce and other functions

DataFrame

edureka!

Inspired by DataFrames in R and Python (Pandas).

DataFrames API is designed to make big data processing on tabular data easier.

DataFrame is a distributed collection of data organized into named columns.

Provides operations to filter, group, or compute aggregates, and can be used with Spark SQL.

Can be constructed from structured data files, existing RDDs, tables in Hive, or external databases.

1. DataFrame in PySpark: Overview

In Apache Spark, a DataFrame is a distributed collection of rows under named columns. In simple terms, it is same as a table in relational database or an Excel sheet with Column headers. It also shares some common characteristics with RDD:

- **Immutable in nature** : We can create DataFrame / RDD once but can't change it. And we can transform a DataFrame / RDD after applying transformations.
- **Lazy Evaluations**: Which means that a task is not executed until an action is performed.
- **Distributed**: RDD and DataFrame both are distributed in nature.

My first exposure to DataFrames was when I learnt about Pandas. Today, it is difficult for me to run my data science workflow with out Pandas DataFrames. So, when I saw similar functionality in Apache Spark, I was excited about the possibilities it opens up!

<https://www.analyticsvidhya.com/blog/2016/10/spark-dataframe-and-operations/>

DataFrame features

edureka!

Ability to scale from KBs to PBs

Support for a wide array of data formats and storage systems

State-of-the-art optimization and code generation through the spark SQL catalyst optimizer

Seamless integration with all big data tooling and infrastructure via spark

APIs for Python, Java, Scala, and R

Ways to Create DataFrame in Spark

Hive Data

Csv Data

Json Data

RDBMS Data

XML Data

Parquet Data

Cassandra Data

RDDs

Spark SQL

DataFrame

	Col1	Col2	Col3	
Row 1				
Row 2				
Row 3				
...				

Web Applications

REST

Full Stack Application

Full Stack Developer Meaning & Definition

In technology development, full stack refers to an entire computer system or application from the **front end** to the **back end** and the **code** that connects the two. The back end of a computer system encompasses "behind-the-scenes" technologies such as the **database** and **operating system**. The front end is the **user interface** (UI). This end-to-end system requires many ancillary technologies such as the **network**, **hardware**, **load balancers**, and **firewalls**.

FULL STACK WEB DEVELOPERS

Full stack is most commonly used when referring to **web developers**. A full stack web developer works with both the front and back end of a website or application. They are proficient in both front-end and back-end **languages** and frameworks, as well as server, network, and **hosting** environments.

Full-stack developers need to be proficient in languages used for front-end development such as **HTML**, **CSS**, **JavaScript**, and third-party libraries and extensions for Web development such as **JQuery**, **SASS**, and **REACT**. Mastery of these front-end programming languages will need to be combined with knowledge of UI design as well as customer experience design for creating optimal front-facing websites and applications.

<https://www.webopedia.com/definitions/full-stack/>

Full Stack Web Developer

A full stack web developer is a person who can develop both **client** and **server** software.

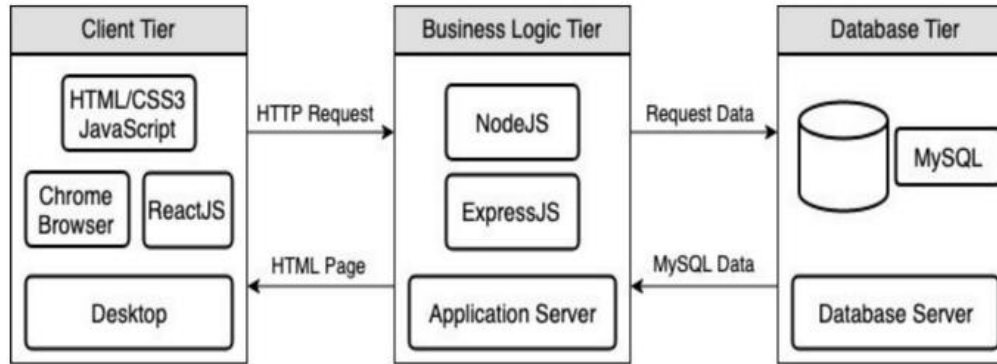
In addition to mastering HTML and CSS, he/she also knows how to:

- Program a **browser** (like using JavaScript, jQuery, Angular, or Vue)
- Program a **server** (like using PHP, ASP, Python, or Node)
- Program a **database** (like using SQL, SQLite, or MongoDB)

https://www.w3schools.com/whatis/whatis_fullstack.asp

- There are courses that cover topics:
 - COMS W4153: Advanced Software Engineering
 - COMS W4111: Introduction to Databases
 - COMS W4170 - User Interface Design
- This course will focus on cloud realization, microservices and application patterns,
- Also, I am not great at UIs We will not emphasize or require a lot of UI work.

Full Stack Web Application



M = Mongo
E = Express
R = React
N = Node

I start with FastAPI and MySQL,
but all the concepts are the same.

<https://levelup.gitconnected.com/a-complete-guide-build-a-scalable-3-tier-architecture-with-mern-stack-es6-ca129d7df805>

- My preferences are to replace React with Angular, and Node with Flask.
- There are three projects to design, develop, test, deploy, ...
 1. Browser UI application.
 2. Microservice.
 3. Database.
- We will initial have two deployments: local machine, virtual machine.
We will ignore the database for step 1.

Some Terms

- A web application server or web application framework: “A web framework (WF) or web application framework (WAF) is a software framework that is designed to support the development of web applications including web services, web resources, and web APIs. Web frameworks provide a standard way to build and deploy web applications on the World Wide Web. Web frameworks aim to automate the overhead associated with common activities performed in web development. For example, many web frameworks provide libraries for database access, templating frameworks, and session management, and they often promote code reuse.” (https://en.wikipedia.org/wiki/Web_framework)
- REST: “REST (Representational State Transfer) is a software architectural style that was created to guide the design and development of the architecture for the World Wide Web. REST defines a set of constraints for how the architecture of a distributed, Internet-scale hypermedia system, such as the Web, should behave. The REST architectural style emphasises uniform interfaces, independent deployment of components, the scalability of interactions between them, and creating a layered architecture to promote caching to reduce user-perceived latency, enforce security, and encapsulate legacy systems.[1]

REST has been employed throughout the software industry to create stateless, reliable web-based applications.” (<https://en.wikipedia.org/wiki/REST>)

Some Terms

- OpenAPI: “The OpenAPI Specification, previously known as the Swagger Specification, is a specification for a machine-readable interface definition language for describing, producing, consuming and visualizing web services.” (https://en.wikipedia.org/wiki/OpenAPI_Specification)
- Model: “A model represents an entity of our application domain with an associated type.” (<https://medium.com/@nicola88/your-first-openapi-document-part-ii-data-model-52ee1d6503e0>)
- Routers: “What fastapi docs says about routers: If you are building an application or a web API, it’s rarely the case that you can put everything on a single file. FastAPI provides a convenience tool to structure your application while keeping all the flexibility.” (<https://medium.com/@rushikeshnaik779/routers-in-fastapi-tutorial-2-adf3e505fdca>)
- Summary:
 - These are general concepts, and we will go into more detail in the semester.
 - FastAPI is a specific technology for Python.
 - There are many other frameworks applicable to Python, NodeJS/TypeScript, Go, C#, Java,
 - They all surface similar concepts with slightly different names.

REST (<https://restfulapi.net/>)

What is REST

- REST is acronym for **RE**presentational **S**tate **T**ransfer. It is architectural style for **distributed hypermedia systems** and was first presented by Roy Fielding in 2000 in his famous [dissertation](#).
- Like any other architectural style, REST also does have it's own [6 guiding constraints](#) which must be satisfied if an interface needs to be referred as **RESTful**. These principles are listed below.

Guiding Principles of REST

- **Client-server** – By separating the user interface concerns from the data storage concerns, server improves its portability and the visibility for developers across multiple platforms and improve scalability by simplifying the server components.
- **Stateless** – Each request from client to server must contain all of the information necessary to understand the request, and to guide the behavior of the server. Session state is therefore kept entirely on the client.
- **Cacheable** – Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- **Uniform interface** – By applying the software engineering principle of generality to the component interface, the overall system architecture is improved. In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state.
- **Layered system** – The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot “see” beyond the immediate layer with which they are interacting.
- **Code on demand (optional)** – REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented.

Resources

Resources are an abstraction. The application maps to create things and actions.

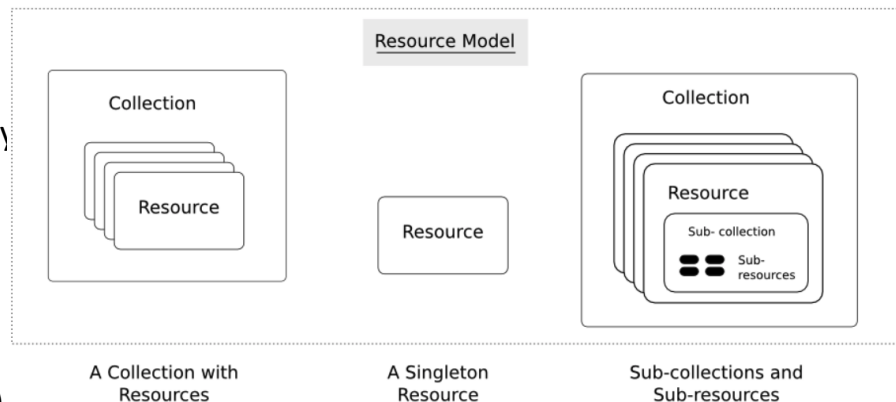
“A resource-oriented API is generally modeled as a resource hierarchy, where each node is either a *simple resource* or a *collection resource*. For convenience, they are often called a resource and a collection, respectively.

- A collection contains a list of resources of **the same type**. For example, a user has a collection of contacts.
- A resource has some state and zero or more sub-resources. Each sub-resource can be either a simple resource or a collection resource.

For example, Gmail API has a collection of users, each user has a collection of messages, a collection of threads, a collection of labels, a profile resource, and several setting resources.

While there is some conceptual alignment between storage systems and REST APIs, a service with a resource-oriented API is not necessarily a database, and has enormous flexibility in how it interprets resources and methods. For example, creating a calendar event (resource) may create additional events for attendees, send email invitations to attendees, reserve conference rooms, and update video conference schedules. (Emphasis added)

(<https://cloud.google.com/apis/design/resources#resources>)



<https://restful-api-design.readthedocs.io/en/latest/resources.html>

REST – Resource Oriented

- When writing applications, we are used to writing functions or methods:
 - `openAccount(last_name, first_name, tax_payer_id)`
 - `account.deposit(deposit_amount)`
 - `account.close()`

We can create and implement whatever functions we need.

- REST only allows four methods:

- POST: Create a resource
- GET: Retrieve a resource
- PUT: Update a resource
- DELETE: Delete a resource

“The key characteristic of a resource-oriented API is that it emphasizes resources (data model) over the methods performed on the resources (functionality). A typical resource-oriented API exposes a large number of resources with a small number of methods.”

(<https://cloud.google.com/apis/design/resources>)

That's it. That's all you get.

- A REST client needs no prior knowledge about how to interact with any particular application or server beyond a generic understanding of hypermedia.

Resources, URLs, Content Types

Accept type in headers.

```
Accept: <MIME_type>/<MIME_subtype>
```

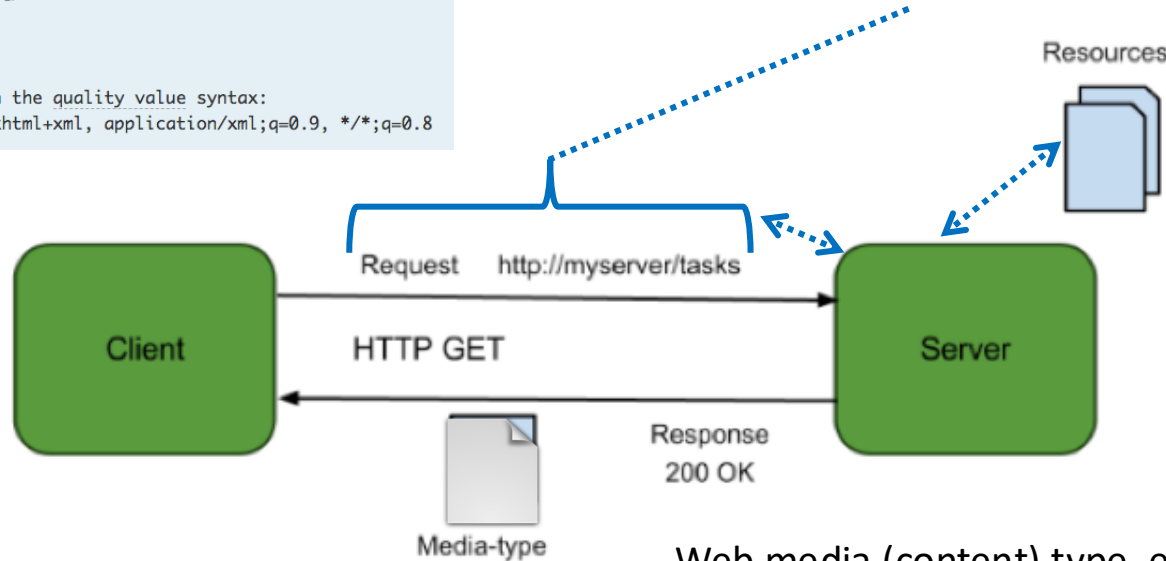
```
Accept: <MIME_type>/*
```

```
Accept: */*
```

```
// Multiple types, weighted with the quality value syntax:
```

```
Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8
```

- Relative URL identifies “resource” on the server.
- Server implementation maps abstract resource to tangible “thing,” file, DB row, ... and any application logic.



Client may be
Browser
Mobile device
Other REST Service
... ..

Web media (content) type, e.g.

- text/html
- application/json

Resources and APIs

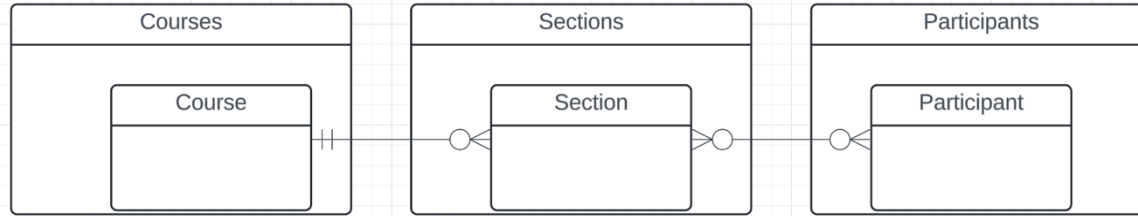
- Base resources

- /courses: GET, POST
- /courses/<id>: GET, PUT, DELETE
- /sections: GET, POST
- /sections/<id>: GET, PUT, DELETE
- /participants: GET, POST
- /participants/<id>: GET, PUT, DELETE

- There are relative, navigation paths:

- /courses/<id>/sections
- /participants/<id>/sections
- etc.

- GET on resources that are collections may also have query parameters.



- There are two approaches to defining API

- Start with OpenAPI document and produce an implementation template.
- Start with annotated code and generate API document.

- In either approach, I start with *models*.

- Also,

- I lack the security permission to update CourseWorks.
- I can choose to not surface the methods or raise and exception.

Data Modeling Concepts and REST

Almost any data model has the same core concepts:

- Types and instances:
 - Entity Type: A definition of a type of thing with properties and relationships.
 - Entity Instance: A specific instantiation of the Entity Type
 - Entity Set Instance: An Entity Type that:
 - Has properties and relationships like any entity, but ...
 - Has at least one *special relationship* – **contains**.
- Operations, minimally CRUD, that manipulate entity types and instances:
 - Create
 - Retrieve
 - Update
 - Delete
 - Reference/Identify/... ..
 - Host/database/table/pk

What is REST architecture?

REST stands for REpresentational State Transfer. REST is web standards based architecture and uses HTTP Protocol. It revolves around resource where every component is a resource and a resource is accessed by a common interface using HTTP standard methods. REST was first introduced by Roy Fielding in 2000.

In REST architecture, a REST Server simply provides access to resources and REST client accesses and modifies the resources. Here each resource is identified by URIs/ global IDs. REST uses various representation to represent a resource like text, JSON, XML. JSON is the most popular one.

HTTP methods

Following four HTTP methods are commonly used in REST based architecture.

- **GET** – Provides a read only access to a resource.
- **POST** – Used to create a new resource.
- **DELETE** – Used to remove a resource.
- **PUT** – Used to update a existing resource or create a new resource.

Introduction to RESTful web services

A web service is a collection of open protocols and standards used for exchanging data between applications or systems. Software applications written in various programming languages and running on various platforms can use web services to exchange data over computer networks like the Internet in a manner similar to inter-process communication on a single computer. This interoperability (e.g., between Java and Python, or Windows and Linux applications) is due to the use of open standards.

Web services based on REST Architecture are known as RESTful web services. These webservices uses HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI, Uniform Resource Identifier a service, provides resource representation such as JSON and set of HTTP Methods.

Creating RESTful Webservice

In next chapters, we'll create a webservice say user management with following functionalities –

Sr.No.	URI	HTTP Method	POST body	Result
1	/UserService/users	GET	empty	Show list of all the users.
2	/UserService/addUser	POST	JSON String	Add details of new user.
3	/UserService/getUser/:id	GET	empty	Show details of a user.